

JAYPEE UNIVERSITY OF ENGINEERING & TECHNOLOGY



**MINOR PROJECT
PRACTICAL-1**

MENTOR- PROF. SANJAY GARG

TEAM MEMBERS

- 1.Ayushmaan Kumar
- 2.Devashish Rawat
- 3.Akshit Shukla

AI-Driven Predictive Traffic Flow Optimization

Urban traffic congestion is worsening due to rapid urbanization, rising vehicle density, and limited road infrastructure. Existing navigation systems are reactive, offering alternate routes only after congestion has formed, leading to longer commutes, fuel wastage, and higher emissions. There is a pressing need for a predictive, AI-driven solution that forecasts congestion and proactively optimizes routes.

Current tools lack advanced machine learning, multi-objective optimization, and integrated data pipelines to support both drivers and urban planners. This project aims to fill that gap by using LSTM neural networks and traffic simulations to provide accurate congestion predictions, optimal route recommendations, and actionable insights for smarter city traffic management.

Tools & Technologies Used

Programming Languages Python	Data Analysis & Processing NumPy, Pandas	Machine Learning & Deep Learning Scikit-learn, TensorFlow, Keras
Data Visualization Matplotlib, Seaborn, Plotly	Simulation Tools SUMO (Simulation of Urban MObility)	Deployment Flask/FastAPI, Docker
Version Control & Collaboration Git, GitHub		

This project introduces a predictive traffic flow optimization system designed to forecast congestion and recommend routes that enhance city-wide traffic patterns. Unlike traditional navigation apps, our system leverages machine learning to anticipate traffic, offering proactive solutions for both individual drivers and urban planners to reduce emissions and commute times.

Our approach differentiates itself through predictive power using LSTM Neural Networks, multi-objective optimization balancing individual and city-wide efficiency, and an innovative data pipeline integrating open datasets and traffic simulations. We aim to provide a foundational solution for smarter urban mobility.

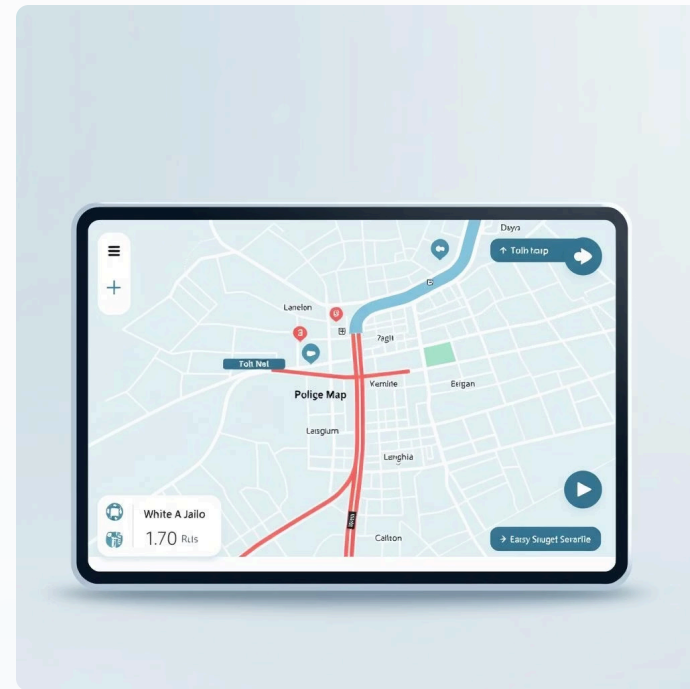
Model Outputs

- Predicts traffic volume congestion levels (e.g., "Downtown area: 85% probability of severe congestion at 6 PM")
- Forecasts average speeds
- Feeds predictions into an optimization engine generating optimal routes (e.g., "18 mins vs. 30 mins on default route")
- Provides heatmaps for city planners to redistribute traffic

Visualizing and Impacting Urban Mobility

Visualizing the Output

- Interactive Maps: Folium and Plotly highlight congestion hotspots
- Web and Mobile Apps: Display predictive routes for drivers
- Dashboards: Enable city planners to monitor traffic trends in real time



Why This Output Matters



Drivers

Save time, reduce fuel consumption, and lower stress during commutes.



Cities

Cut emissions, improve road safety, and support infrastructure planning.



Businesses

Optimize logistics and delivery routes for efficiency and cost savings.

This project develops a predictive traffic flow optimization system using machine learning to forecast congestion. By analyzing historical data and real-time factors, it predicts traffic volume and speed using LSTM models. The system, deployed as a web application with interactive maps, addresses urban traffic challenges, reduces emissions, and improves commute efficiency.

Project Outcomes and Dataset Overview

Expected Outcome (End of 5th Semester)

- Trained ML model with >85% accuracy predicting hourly traffic volume
- Functional web application (Flask API) for route optimization and congestion visualization
- Simulation results (SUMO) showing 15,320% reduction in average commute time
- Comprehensive documentation and research paper



Bangalore Traffic Dataset

This dataset contains 8,936 records across 16 columns, ideal for traffic flow optimization projects.

Dataset Details

- Records: 8,936
- Columns: 16
- Use Case: Traffic Flow Optimization

Key Numerical Statistics

- Traffic Volume: 4,233 to 72,039 vehicles
- Average Speed: 20 to 90 km/h (Mean ~39.4)
- Congestion Level: Often 100%
- Road Capacity Utilization: Peaks at 100% (Mean ~92%)
- Incident Reports: Typically 0.33 per location/day
- Environmental Impact: Mean ~108 (emission models)
- Public Transport Usage: Mean ~45%
- Pedestrian/Cyclist Count: 66 to 243 people

Categorical Features

- Area Name: 8 unique areas (e.g., Indiranagar, Koramangala)
- Road/Intersection Name: 16 unique locations
- Weather Conditions: 5 types, predominantly "Clear"
- Roadwork Activity: Mostly "No" (~90% of rows)

All Column Names (16 Total)

Traffic Volume, Area Name, Average Speed, Road/Intersection Name, Congestion Level, Weather Conditions, Road Capacity Utilization, Roadwork Activity, Incident Reports, Timestamp, Environmental Impact, Vehicle Type, Public Transport Usage, Direction, Pedestrian/Cyclist Count, Light Condition

Why Are We Using LSTM?

We train an LSTM (Long Short-Term Memory) model to predict traffic congestion levels based on historical data. LSTMs are a type of Recurrent Neural Network (RNN) that excel at learning patterns in sequences, such as time-series data.

In our case, the "sequence" is congestion levels over time, and the model predicts the future congestion level. Think of it like teaching the model: "Here's congestion for the last 6 time steps; now guess the next one."

For Our Project, LSTM Learns Patterns Like:

- If congestion has been rising for 6 steps, it's likely to keep rising.
- After peak hours, congestion usually falls.

It can capture both short-term trends (last few minutes) and long-term cycles (rush-hour patterns).

Why Not Use a Normal Neural Network?

- A normal feed-forward network treats each input independently.
- It doesn't remember what happened before, so it can't use historical context.
- Example: If you give it "Congestion is 60 right now," it doesn't know it has been increasing steadily for the last hour.

Understanding LSTM Networks

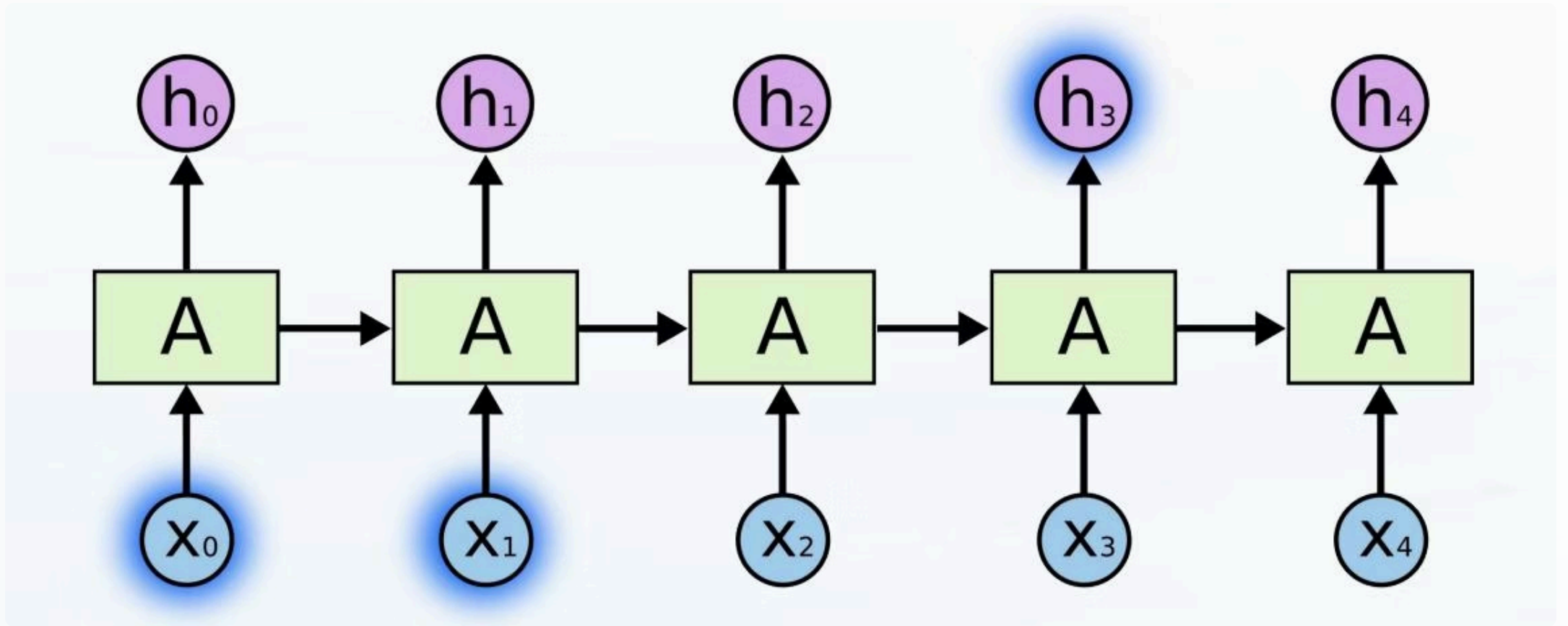
Unlike traditional neural networks, Recurrent Neural Networks (RNNs) have loops, allowing information to persist and influence subsequent steps. This persistence is crucial for tasks requiring context from previous data, such as understanding sequences in language or video.

RNNs can be visualized as multiple copies of the same network, each passing a message along a chain. This structure makes them ideal for sequential data like speech, text, and time series.

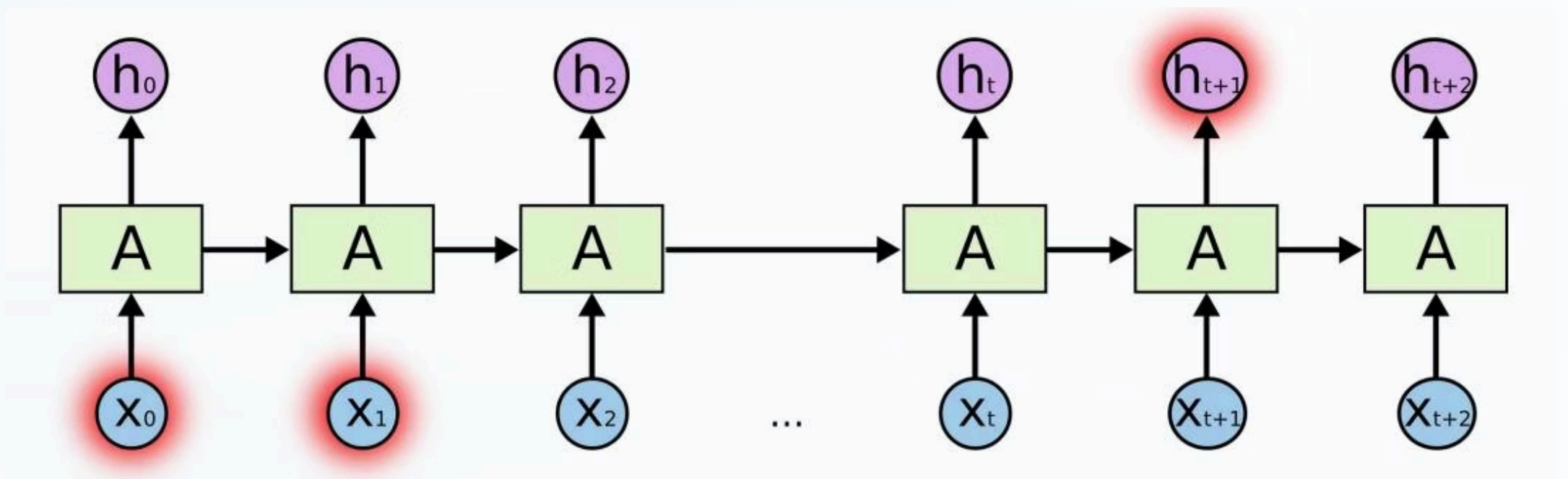
Recurrent Neural Networks

The Challenge of Long-Term Dependencies

A key appeal of RNNs is their ability to connect past information to current tasks. While they excel at short-term dependencies (e.g., predicting "sky" in "clouds are in the sky"), they struggle with long-term dependencies.



Short-Term Context



RNNs can easily use recent information for predictions.

Long-Term Challenge

As the gap between relevant information and its need grows, RNNs struggle to learn connections. This limitation was explored by researchers like Hochreiter (1991) and Bengio et al. (1994), highlighting fundamental reasons for this difficulty.

Introducing LSTM Networks

Long Short-Term Memory (LSTM) networks are a specialized type of RNN designed to overcome the long-term dependency problem. Introduced by Hochreiter & Schmidhuber (1997), LSTMs are now widely used due to their exceptional performance across various tasks.

Learning Long-Term Dependencies

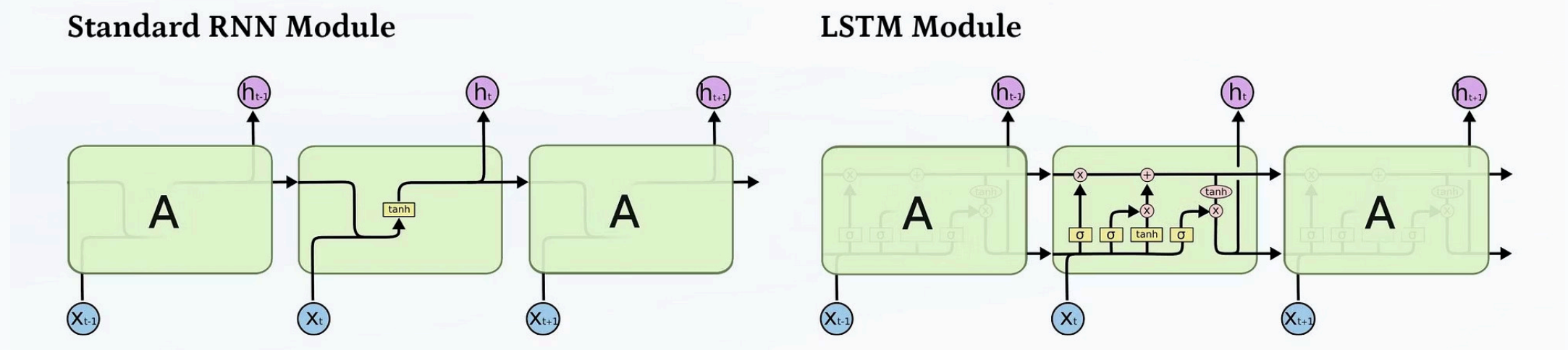
LSTMs are explicitly designed to remember information over extended periods. They significantly outperform standard RNNs in most applications.

Default Behavior

Remembering information is a core, inherent capability of LSTMs.

LSTM Architecture

While standard RNNs use a simple repeating module (e.g., a single tanh layer), LSTMs feature a more complex structure within their repeating module.

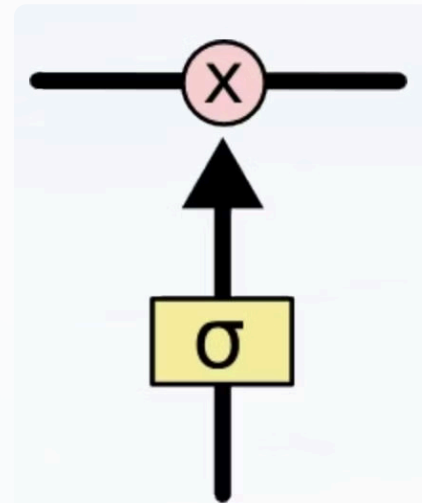
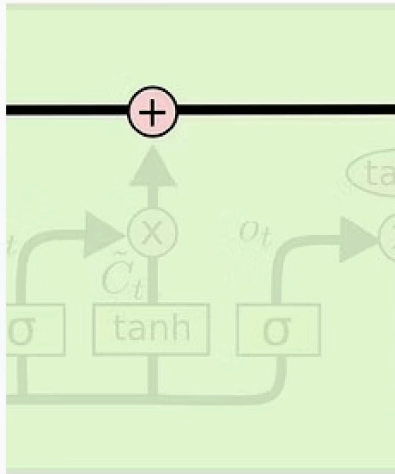


The LSTM module contains four interacting neural network layers, allowing for sophisticated control over information flow

The Core Idea: Cell State & Gates

The central component of an LSTM is the **cell state**, a horizontal line that acts like a conveyor belt, allowing information to flow through the network largely unchanged.

LSTMs use **gates** to regulate the flow of information into and out of the cell state. These gates are composed of a sigmoid neural net layer and a pointwise multiplication operation, outputting values between 0 and 1 to control how much information is passed through.

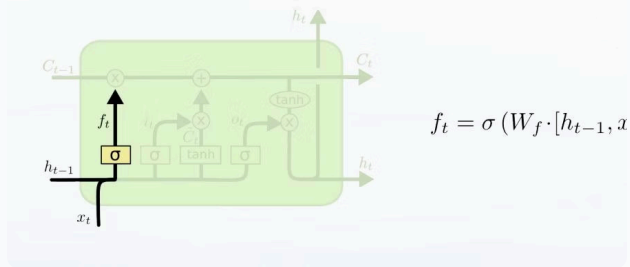


LSTM Gates in Action

1

Forget Gate Layer

Decides what information to discard from the cell state, updating it by combining old state with new information.



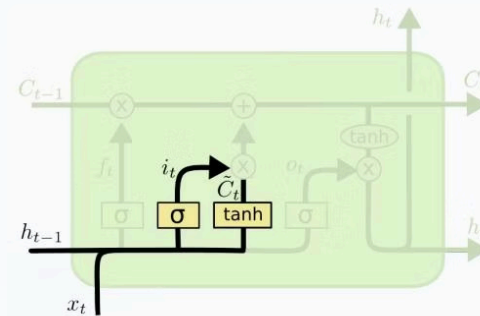
$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

2

Input Gate Layer

Determines which new information to store in the cell state.

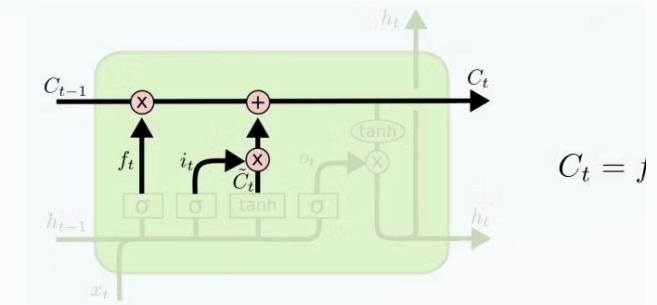


$$i_t =$$
$$\tilde{C}_t =$$

3

Output Gate Layer

Decides what parts of the cell state to output.



Data Preprocessing Pipeline

The `preprocess_data()` function is the main pipeline that prepares our dataset for analysis and machine learning.

Find Dataset

Locates the dataset file inside the project directory.

Load Data

Loads the dataset into memory as a table using pandas.

Structure Data

Keeps only the expected columns to maintain consistent dataset structure.

Create Features

Generates new time-based features like weekday, weekend, month, and year from the date column.

Handle Missing Data

Fills missing values with suitable replacements such as median or mean.

Normalize Features

Scales important numerical features between 0 and 1 for model compatibility.

Return Results

Outputs the processed dataset and scaler object for consistent scaling on test data.

`convert_datetime_features(df)`

This function enriches the dataset by adding time-related features such as year, month, and day, helping models detect temporal patterns.



Convert Date Format

It first ensures the "Date" column is actually in a date format (instead of plain text).



Extract Time Features

From that date, it extracts useful information:

- Day of the week (to capture weekday vs weekend patterns).
- Weekend flag (a simple yes/no to indicate if the day is Saturday or Sunday).
- Month (to capture seasonal changes).

Think of this as the essential data cleaning and preparation pipeline that enables effective machine learning.